

数据库系统 project 报告

2025-2026 学年第 2 学期（CST21118）

数据库系统 project 任务书	
名称	OpenGauss 数据库应用开发
类型	☐ 验证性 ☐ 设计性 ☒ 综合性
内容	设计并编程实现一个小型的企业员工管理系统。 （1）对企业员工管理系统进行需求分析，设计 E-R 模型，详细描述实体的属性和实体之间的联系，消除不必要的冗余；实现 E-R 图向关系模型的转换，定义主键、外键，同时设计出系统功能模块图。 （2）基于 OpenGauss 数据库搭建企业员工管理系统数据库。 通过 Python、java 等程序设计语言编程实现企业员工管理系统，重点是实现数据库的连接和各种 CRUD 操作；必须有对数据的增、删、改、查功能。界面自选，可以是命令行形式，web 页面形式或者 GUI。
要求	1. 设计方案要合理； 2. 能基于该方案完成系统要求的功能； 3. 设计方案有一定的合理性分析。
任务时间	2026 年 4 月 20 日至 2026 年 5 月 10 日

小组成员			
20240395	20240640	学号	学号
严浩睿	马沛霖	姓名	姓名
项目评分表			
序号	评分项	比例	得分
1	Project 内容完成情况	50%	
2	工具熟练度	30%	
3	团队协作	20%	
项目总得分：			

课程项目评分标准（总分 10 分）

考核内容 (权重)	评分标准			
	10	8-9	6-7	0-5（不合格）
1. 完成 project 的实现（50%）	按时完成 project 分析、设计、实现等核心内容。	完成 project 分析、设计、实现等核心内容，但存在少量错误。	基本 project 分析、设计、实现等核心内容，但存在较多错误。	未完成 project 内容。
2. 熟练使用设计工具（30%）	熟练掌握实验设计相关的软件工具，完成数据库设计、编码等工作。	较为掌握实验设计相关的软件工具，完成数据库设计、编码等工作。	能够使用实验设计相关的软件工具，完成数据库设计、编码等工作。	对于实验设计软件和工具使用不熟练。
3. 团队协作（20%）	有团队，分工合理，密切协作。	有团队，分工合理，有一定协作。	有团队，分工不合理，无协作。	无团队，无协作。

Contents

小组分工	1
1 企业员工管理系统背景分析	1
1.1 项目背景	1
1.2 系统目标	2
1.3 需求分析	2
1.4 系统功能模块设计	3
2 企业员工管理系统概念设计	5
2.1 实体识别	5
2.2 实体属性	5
2.3 实体间联系	6
2.4 E-R 图	6
3 企业员工管理系统的关系模型及优化	7
3.1 E-R 图向关系模型的转换	7
3.2 关系模型优化	8
4 企业员工管理系统数据库物理设计	9
4.1 数据库创建	9
4.2 数据表设计	9
4.2.1 部门表(Department)	10
4.2.2 岗位表(Position)	10
4.2.3 员工表(Employee)	11
4.2.4 薪资表(Salary)	11
4.2.5 考勤表(Attendance)	12
4.3 建表 SQL 语句	12
4.4 索引设计	14
5 程序设计语言与数据库的连接与操作	15
5.1 开发环境	15
5.2 数据库连接	17

5.3	数据插入操作(Create)	18
5.4	数据查询操作(Read)	20
5.5	数据更新操作(Update)	21
5.6	数据删除操作>Delete)	22
5.7	系统界面展示	25
总结		28

小组分工

本项目由两名成员合作完成，具体分工如下表所示。在项目开发过程中，两位成员密切配合，定期进行进度同步和技术讨论，确保项目按时高质量完成。

Table 1: 小组成员及分工

学号	姓名	负责内容
20240395	严浩睿	需求分析、E-R 图设计、关系模型设计、数据库建表与优化
20240640	马沛霖	数据库物理设计、程序编码实现、数据库连接与 CRUD 操作

协作说明：严浩睿同学主要负责系统前期的需求调研和数据库设计工作，包括业务流程分析、E-R 模型构建、关系模式转换及数据库表结构设计。马沛霖同学主要负责数据库的物理实现和应用程序开发，包括 OpenGauss 数据库部署、SQL 语句编写、Python 程序开发及系统测试。两位同学在项目关键节点进行了充分的技术交流和方案讨论。

1 企业员工管理系统背景分析

1.1 项目背景

随着企业规模的不断扩大和信息化建设的深入推进，传统的纸质档案管理和人工统计方式已无法满足现代企业对员工信息管理的需求。企业员工信息涉及个人基本信息、部门、岗位、薪资、考勤等多个维度，数据量大且更新频繁，亟需一套高效、可靠的管理系统来实现对员工信息的集中管理和快速查询。

从行业发展角度来看,人力资源管理信息化已成为企业数字化转型的重要组成部分。根据相关调研数据显示,超过 80% 的中大型企业已采用或计划采用电子化的人力资源管理系统。这些系统不仅能够提高人事部门的工作效率,还能为管理层提供数据支持,辅助决策制定。

本系统基于国产数据库 OpenGauss,设计并实现一个小型的企业员工管理系统。OpenGauss 是由华为开源的企业级关系型数据库,具有高性能、高可用、高安全等特点,适用于各类企业应用场景。选择 OpenGauss 作为本系统的数据库平台,主要基于以下考虑:

- (1) **技术先进性:** OpenGauss 基于 PostgreSQL 发展而来,支持丰富的数据类型和先进的查询优化技术;
- (2) **自主可控:** 作为国产开源数据库,OpenGauss 在信息安全方面具有优势,符合国家信创战略要求;
- (3) **生态完善:** OpenGauss 提供了完善的开发工具和管理平台,便于系统开发和维护;
- (4) **性能优异:** 针对企业级应用场景进行了深度优化,能够支持高并发和大容量数据处理。

1.2 系统目标

本系统旨在实现以下目标:

- (1) **数据集中管理:** 建立规范化的企业员工信息数据库,实现员工信息的统一存储和管理,避免数据分散和重复;
- (2) **功能完善:** 提供完善的数据增删改查(CRUD)功能,满足日常员工信息管理需求,包括员工入职、调岗、离职等全生命周期管理;
- (3) **数据质量保障:** 确保数据的一致性和完整性,通过主键、外键、检查约束等数据库机制减少数据冗余和异常;
- (4) **易用性:** 提供友好的用户交互界面,降低使用门槛,使非技术人员也能轻松操作系统;
- (5) **可扩展性:** 系统架构设计考虑未来功能扩展需求,便于后续添加绩效考核、培训管理等模块。

1.3 需求分析

通过对企业员工管理业务的深入分析，本系统需要管理以下核心信息：

- **员工信息管理**：员工的基本信息（姓名、性别、出生日期、联系方式、入职日期等）的录入、修改、删除和查询。需要支持按部门、岗位、入职时间等多条件筛选。
- **部门信息管理**：部门的创建、修改、删除和查询，以及部门与员工之间的归属关系。需要统计各部门人员数量，支持组织架构图展示。
- **岗位信息管理**：岗位的设置、调整和查询，以及岗位与员工的对应关系。需要管理岗位职责描述和薪资等级。
- **薪资信息管理**：员工薪资的录入、调整和查询，包括基本工资、绩效、奖金等。需要支持按月查询和统计报表生成。
- **考勤信息管理**：员工出勤记录的录入、统计和查询。需要支持正常、迟到、早退、请假、缺勤等多种状态记录。

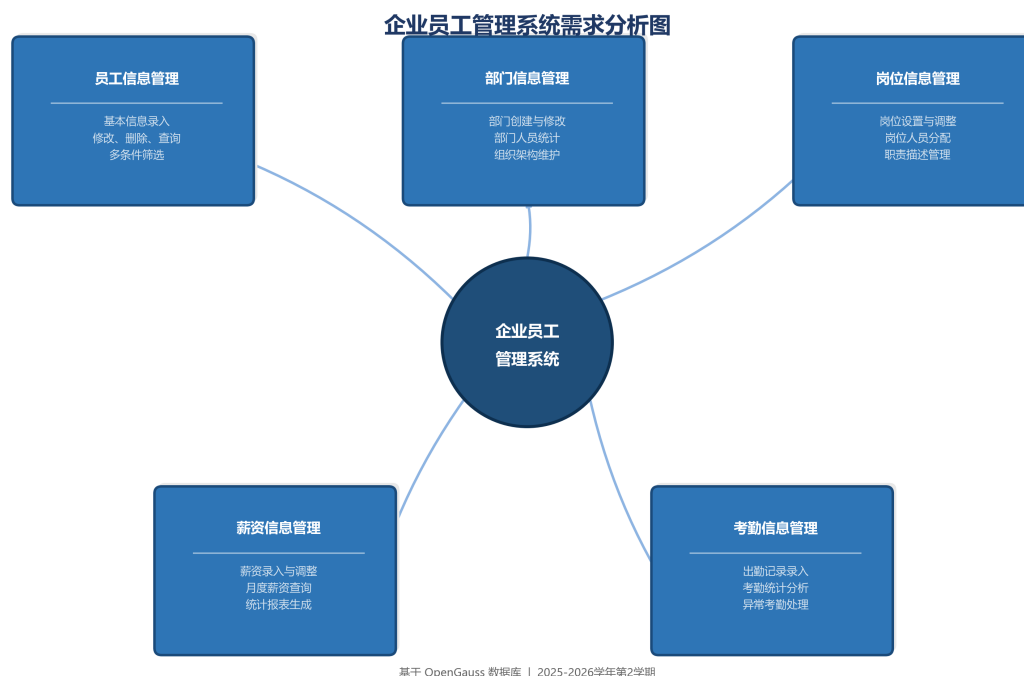


Figure 1: 企业员工管理系统需求分析图

图1展示了系统的五大核心需求模块及其功能要点。每个模块都围绕员工管理的特定业务场景设计，模块之间通过员工实体相互关联，形成完整的业务闭环。

1.4 系统功能模块设计

基于上述需求分析，本系统划分为以下功能模块：

Table 2: 系统功能模块

模块编号	模块名称	功能描述
M1	员工管理模块	员工信息的增删改查,支持多条件筛选,员工档案管理
M2	部门管理模块	部门信息的增删改查,部门人员统计,组织架构维护
M3	岗位管理模块	岗位信息的增删改查,岗位人员分配,职责描述管理
M4	薪资管理模块	薪资信息的录入、调整和查询统计,薪资报表生成
M5	考勤管理模块	考勤记录的录入、统计和报表生成,异常考勤处理

各模块之间的业务关系如下：员工管理模块是系统的核心，与其他所有模块都存在数据关联。部门管理模块和岗位管理模块为员工管理提供基础数据支持。薪资管理模块和考勤管理模块则基于员工信息进行业务数据记录和分析。

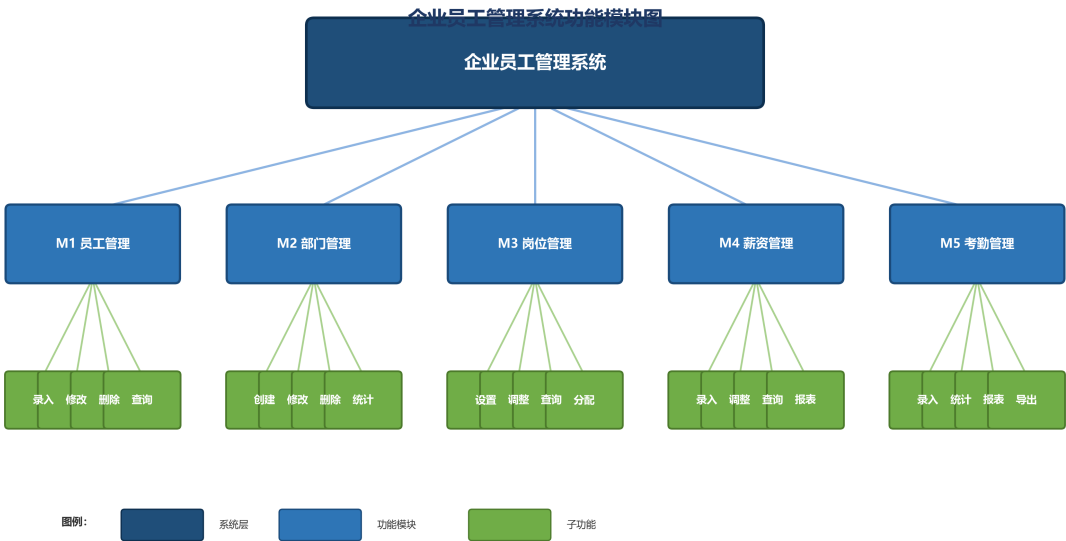


Figure 2: 企业员工管理系统功能模块图

图2以树状结构展示了系统的功能模块层次关系。系统层包含五个一级功能模块，每

个一级模块下又细分为若干子功能,形成了清晰的功能架构。

2 企业员工管理系统概念设计

2.1 实体识别

通过对系统需求的分析,识别出以下核心实体:

1. **员工(Employee)**:系统中的核心实体,记录每位员工的个人信息。员工是企业的基本组成单位,所有其他业务数据都围绕员工展开。
- 2.
3. **部门(Department)**:企业的组织单位,与员工存在归属关系。部门反映了企业的组织架构,是员工管理的重要维度。
4. **岗位(Position)**:员工的职务岗位,与员工存在对应关系。岗位决定了员工的职责范围和薪资等级。
5. **薪资(Salary)**:员工的薪资记录,与员工存在关联。薪资是员工报酬的量化体现,需要按月记录和管理。
6. **考勤(Attendance)**:员工的出勤记录,与员工存在关联。考勤反映了员工的工作状态,是薪资计算的重要依据。

2.2 实体属性

各实体的主要属性如下表所示:

Table 3: 实体及其属性

实体	主键	主要属性
员工	emp_id	姓名、性别、出生日期、电话、邮箱、入职日期
部门	dept_id	部门名称、部门描述、负责人
岗位	pos_id	岗位名称、岗位描述、薪资等级
薪资	salary_id	基本工资、绩效工资、奖金、发放月份
考勤	attend_id	考勤日期、出勤状态、备注

属性设计说明:

- 所有实体都采用自增整数作为主键,确保记录的唯一性和查询效率;
- 员工实体的邮箱属性设置为 UNIQUE 约束,保证邮箱地址不重复;
- 性别属性采用单字符存储(M/F),节省存储空间;
- 薪资和考勤实体通过外键关联到员工实体,建立数据关联关系。

2.3 实体间联系

- **部门—员工**: 1 对多联系。一个部门可以有多名员工,一名员工属于一个部门。这种联系反映了企业的组织架构,通过 Employee 表中的 dept_id 外键实现。
- **岗位—员工**: 1 对多联系。一个岗位可以有多名员工,一名员工担任一个岗位。这种联系体现了岗位与人员的对应关系,通过 Employee 表中的 pos_id 外键实现。
- **员工—薪资**: 1 对多联系。一名员工可以有多条薪资记录(按月发放)。这种联系支持薪资的历史查询和趋势分析。
- **员工—考勤**: 1 对多联系。一名员工可以有多条考勤记录(按日记录)。这种联系支持考勤的詳細统计和报表生成。

2.4 E-R 图

根据以上分析,绘制出系统的 E-R 图如图3所示。



Figure 3: 企业员工管理系统 E-R 图

E-R 图说明：

- 矩形(蓝色)表示实体：员工、部门、岗位、薪资、考勤；
- 椭圆(浅蓝)表示属性，金色椭圆(带下划线)表示主键属性；
- 菱形(绿色)表示联系：属于、担任、拥有、记录；
- 连线上的数字(1、N)表示联系的基数，均为一对多(1:N)联系。

联系详解：

- 部门-1-属于-N-员工：一个部门有多名员工，一名员工属于一个部门；
- 岗位-1-担任-N-员工：一个岗位有多名员工，一名员工担任一个岗位；
- 员工-1-拥有-N-薪资：一名员工按月有多条薪资记录；
- 员工-1-记录-N-考勤：一名员工按日有多条考勤记录。

3 企业员工管理系统的关系模型及优化

3.1 E-R 图向关系模型的转换

根据 E-R 图，将实体和联系转换为以下关系模式：

1. **Department**(dept_id, dept_name, description, manager)
2. **Position**(pos_id, pos_name, description, salary_level)
3. **Employee**(emp_id, emp_name, gender, birth_date, phone, email, hire_date, *dept_id*, *pos_id*)
4. **Salary**(salary_id, *emp_id*, base_salary, perf_bonus, bonus, pay_month)
5. **Attendance**(attend_id, *emp_id*, attend_date, status, remark)

其中，下划线表示主键，斜体表示外键。

转换规则说明：

- 每个实体转换为一个关系模式，实体的属性即为关系的属性；

- 实体的主键转换为关系的主键；
- 1:N 联系通过在外键端(N 端)添加外键属性实现,外键引用 1 端的主键；
- 所有联系都转换为外键约束,确保引用完整性。

3.2 关系模型优化

对上述关系模型进行规范化分析：

1NF 分析：所有关系的每个属性都是不可再分的原子值，满足第一范式。例如，员工姓名是一个完整的字符串，没有进一步分解的必要；地址信息虽然可以分解为省、市、区等，但本系统不需要如此详细的地址管理，因此保持原子性即可。

2NF 分析：各关系模式中的非主属性都完全依赖于主键，不存在部分函数依赖，满足第二范式。以 Employee 关系为例，所有非主属性（emp_name、gender 等）都完全依赖于主键 emp_id，不存在只依赖于主键某一部分的情况。

3NF 分析：各关系模式中不存在非主属性对主键的传递依赖，满足第三范式。例如，Department 关系中的 manager 属性直接依赖于 dept_id，不存在通过其他属性间接依赖的情况。

优化措施：

- **消除冗余：**将部门负责人的信息从 Department 表中独立，通过 Employee 表的 dept_id 外键关联来体现，避免了数据冗余。
- **明确联系：**在 Employee 表中设置 dept_id 和 pos_id 外键，明确了部门与员工的 1:N 联系和岗位与员工的 1:N 联系，减少了冗余数据。
- **保证完整性：**薪资表和考勤表通过 emp_id 外键关联到员工表，保证了数据的一致性和引用完整性。
- **索引优化：**为常用查询字段创建索引，提高查询效率。

图4展示了系统的完整关系模型，包括五个关系模式的结构和它们之间的外键关联关系。图中用不同颜色区分了主键（金色）和外键（绿色），红色箭头表示外键引用方向。

企业员工管理系统关系模型图

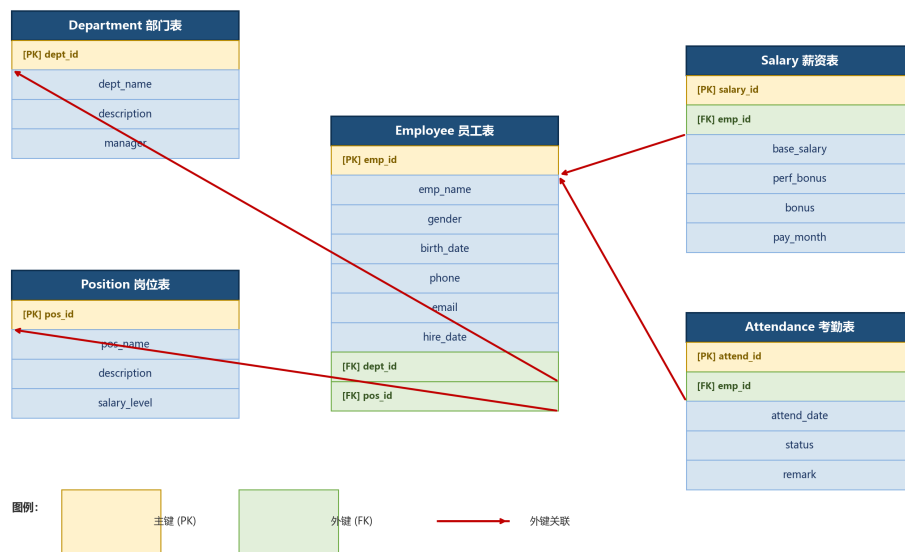


Figure 4: 企业员工管理系统关系模型图(PK= 主键,FK= 外键,红色箭头 = 外键关联)

4 企业员工管理系统数据库物理设计

4.1 数据库创建

在 OpenGauss 数据库中创建企业员工管理系统的数据库,SQL 语句如下:

Listing 1: 创建数据库

```

1  -- 创建数据库
2  CREATE DATABASE employee_db;

```

图5展示了在 OpenGauss 中创建数据库和序列的执行过程。序列 (Sequence) 用于自动生成主键值,确保每条记录的唯一性。本系统为每个表都创建了对应的序列,如 emp_seq、dept_seq 等。

4.2 数据表设计

根据关系模型,设计以下数据表:

```

postgres=# CREATE DATABASE employee_db;
CREATE DATABASE
postgres=# \c employee_db;
Non-SSL connection (SSL connection is recommended when requiring high-security)
You are now connected to database "employee_db" as user "omm".
employee_db=# CREATE SEQUENCE emp_seq START 1;
CREATE SEQUENCE
employee_db=# CREATE SEQUENCE dept_seq START 1;
CREATE SEQUENCE
employee_db=# CREATE SEQUENCE pos_seq START 1;
CREATE SEQUENCE
employee_db=# CREATE SEQUENCE salary_seq START 1;
CREATE SEQUENCE
employee_db=# CREATE SEQUENCE attend_seq START 1;
CREATE SEQUENCE

```

Figure 5: 创建数据库 employee_db 及序列

Table 4: Department 表结构

字段名	数据类型	约束	说明
dept_id	INT	PRIMARY KEY	部门编号, 主键, 自增
dept_name	VARCHAR(50)	NOT NULL	部门名称, 不允许为空
description	VARCHAR(200)		部门描述, 可选
manager	VARCHAR(50)		部门负责人, 可选

4.2.1 部门表(Department)

设计说明: 部门表是系统的基础数据表之一, 存储企业的组织架构信息。dept_name 设置为 NOT NULL 约束, 确保每个部门都有名称。description 和 manager 字段允许为空, 因为新创建的部门可能暂时没有详细描述或负责人。

4.2.2 岗位表(Position)

Table 5: Position 表结构

字段名	数据类型	约束	说明
pos_id	INT	PRIMARY KEY	岗位编号, 主键, 自增
pos_name	VARCHAR(50)	NOT NULL	岗位名称, 不允许为空
description	VARCHAR(200)		岗位描述, 可选
salary_level	INT		薪资等级, 1-10 级

设计说明: 岗位表管理企业的职位体系。salary_level 字段用于关联薪资标准, 便于后续薪资计算和管理。

4.2.3 员工表(Employee)

Table 6: Employee 表结构

字段名	数据类型	约束	说明
emp_id	INT	PRIMARY KEY	员工编号, 主键, 自增
emp_name	VARCHAR(50)	NOT NULL	员工姓名, 不允许为空
gender	CHAR(1)	CHECK (gender IN ('M','F'))	性别, M 男/F 女
birth_date	DATE		出生日期, 可选
phone	VARCHAR(20)		联系电话, 可选
email	VARCHAR(100)	UNIQUE	电子邮箱, 唯一约束
hire_date	DATE	NOT NULL	入职日期, 不允许为空
dept_id	INT	FOREIGN KEY	所属部门, 外键引用 Department
pos_id	INT	FOREIGN KEY	岗位, 外键引用 Position

设计说明: 员工表是系统的核心表, 包含员工的基本信息和组织关系。gender 字段使用 CHECK 约束确保只能输入'M' 或'F'; email 字段设置 UNIQUE 约束保证邮箱不重复; dept_id 和 pos_id 作为外键, 建立与部门表和岗位表的关联关系。

4.2.4 薪资表(Salary)

Table 7: Salary 表结构

字段名	数据类型	约束	说明
salary_id	INT	PRIMARY KEY	薪资记录编号, 主键, 自增
emp_id	INT	FOREIGN KEY	员工编号, 外键引用 Employee
base_salary	DECIMAL(10,2)	NOT NULL	基本工资, 不允许为空
perf_bonus	DECIMAL(10,2)	DEFAULT 0	绩效工资, 默认 0
bonus	DECIMAL(10,2)	DEFAULT 0	奖金, 默认 0
pay_month	VARCHAR(7)	NOT NULL	发放月份 (YYYY-MM)

设计说明: 薪资表记录员工每月的薪资发放情况。perf_bonus 和 bonus 字段设置 DEFAULT 0, 表示如果没有绩效或奖金则默认为 0。pay_month 字段采用 VARCHAR(7) 类型存储'YYYY-MM' 格式的月份信息。

4.2.5 考勤表(Attendance)

Table 8: Attendance 表结构

字段名	类型	约束	说明
attend_id	INT	PRIMARY KEY	考勤记录编号, 主键, 自增
emp_id	INT	FOREIGN KEY	员工编号, 外键引用 Employee
attend_date	DATE	NOT NULL	考勤日期, 不允许为空
status	VARCHAR(10)	CHECK	出勤状态(正常/迟到/早退/请假/缺勤)
remark	VARCHAR(200)	-	备注, 可选

设计说明: 考勤表记录员工每日的出勤情况。status 字段使用 CHECK 约束限制只能输入五种预定义状态, 确保数据规范性。remark 字段用于记录特殊情况说明, 如迟到原因、请假类型等。

4.3 建表 SQL 语句

Listing 2: 创建部门表

```
1 CREATE TABLE Department (  
2     dept_id      INT PRIMARY KEY,  
3     dept_name    VARCHAR(50) NOT NULL,  
4     description  VARCHAR(200),  
5     manager      VARCHAR(50)  
6 );
```

Listing 3: 创建岗位表

```
1 CREATE TABLE Position (  
2     pos_id       INT PRIMARY KEY,  
3     pos_name     VARCHAR(50) NOT NULL,  
4     description  VARCHAR(200),  
5     salary_level INT
```



```

employee_db=# CREATE SEQUENCE attend_seq START 1;
CREATE SEQUENCE
employee_db=# CREATE TABLE Department (
employee_db(#      dept_id      INT PRIMARY KEY DEFAULT nextval('dept_seq'),
employee_db(#      dept_name    VARCHAR(50) NOT NULL,
employee_db(#      description  VARCHAR(200),
employee_db(#      manager     VARCHAR(50)
employee_db(# );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "department_pkey" for table "department"
CREATE TABLE

```

Figure 6: Department 表创建结果

```

6 );

```

```

employee_db=# CREATE TABLE Position (
employee_db(#      pos_id      INT PRIMARY KEY DEFAULT nextval('pos_seq'),
employee_db(#      pos_name    VARCHAR(50) NOT NULL,
employee_db(#      description  VARCHAR(200),
employee_db(#      salary_level INT
employee_db(# );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "position_pkey" for table "position"
CREATE TABLE

```

Figure 7: Position 表创建结果

Listing 4: 创建员工表

```

1 CREATE TABLE Employee (
2     emp_id      INT PRIMARY KEY,
3     emp_name    VARCHAR(50) NOT NULL,
4     gender      CHAR(1) CHECK (gender IN ('M', 'F')),
5     birth_date  DATE,
6     phone       VARCHAR(20),
7     email       VARCHAR(100) UNIQUE,
8     hire_date   DATE NOT NULL,
9     dept_id     INT REFERENCES Department(dept_id),
10    pos_id      INT REFERENCES Position(pos_id)
11 );

```

Listing 5: 创建薪资表

```

1 CREATE TABLE Salary (
2     salary_id   INT PRIMARY KEY,
3     emp_id      INT REFERENCES Employee(emp_id),
4     base_salary DECIMAL(10,2) NOT NULL,
5     perf_bonus  DECIMAL(10,2) DEFAULT 0,

```

```

employee_db=# CREATE TABLE Employee (
employee_db(#      emp_id      INT PRIMARY KEY DEFAULT nextval('emp_seq'),
employee_db(#      emp_name    VARCHAR(50) NOT NULL,
employee_db(#      gender      CHAR(1) CHECK (gender IN ('M', 'F')),
employee_db(#      birth_date  DATE,
employee_db(#      phone       VARCHAR(20),
employee_db(#      email       VARCHAR(100) UNIQUE,
employee_db(#      hire_date   DATE NOT NULL,
employee_db(#      dept_id     INT REFERENCES Department(dept_id),
employee_db(#      pos_id      INT REFERENCES Position(pos_id)
employee_db(#      );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "employee_pkey" for table "employee"
NOTICE: CREATE TABLE / UNIQUE will create implicit index "employee_email_key" for table "employee"
CREATE TABLE

```

Figure 8: Employee 表创建结果

```

6      bonus          DECIMAL(10,2) DEFAULT 0,
7      pay_month      VARCHAR(7) NOT NULL
8  );

```

```

employee_db=# CREATE TABLE Salary (
employee_db(#      salary_id   INT PRIMARY KEY DEFAULT nextval('salary_seq'),
employee_db(#      emp_id      INT REFERENCES Employee(emp_id),
employee_db(#      base_salary DECIMAL(10,2) NOT NULL,
employee_db(#      perf_bonus  DECIMAL(10,2) DEFAULT 0,
employee_db(#      bonus       DECIMAL(10,2) DEFAULT 0,
employee_db(#      pay_month   VARCHAR(7) NOT NULL
employee_db(#      );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "salary_pkey" for table "salary"
CREATE TABLE

```

Figure 9: Salary 表创建结果

Listing 6: 创建考勤表

```

1 CREATE TABLE Attendance (
2     attend_id    INT PRIMARY KEY,
3     emp_id       INT REFERENCES Employee(emp_id),
4     attend_date  DATE NOT NULL,
5     status       VARCHAR(10) CHECK(status IN ('正常', '迟到', '早退', '请
6         假', '缺勤')),
7     remark       VARCHAR(200)
8 );

```

4.4 索引设计

为了提高查询效率,在以下字段上创建索引:

```
employee_db=# CREATE TABLE Attendance (  
employee_db(#      attend_id  INT PRIMARY KEY DEFAULT nextval('attend_seq'),  
employee_db(#      emp_id     INT REFERENCES Employee(emp_id),  
employee_db(#      attend_date DATE NOT NULL,  
employee_db(#      status     VARCHAR(10) CHECK (status IN ('正常','迟到','早退','请假','缺勤')),  
employee_db(#      remark     VARCHAR(200)  
employee_db(#      );  
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "attendance_pkey" for table "attendance"  
CREATE TABLE
```

Figure 10: Attendance 表创建结果

Listing 7: 创建索引

```
1  -- 员工姓名索引（加速按姓名查询）  
2  CREATE INDEX idx_emp_name ON Employee(emp_name);  
3  
4  -- 员工部门索引（加速按部门查询员工）  
5  CREATE INDEX idx_emp_dept ON Employee(dept_id);  
6  
7  -- 薪资员工索引（加速按员工查询薪资）  
8  CREATE INDEX idx_salary_emp ON Salary(emp_id);  
9  
10 -- 考勤员工和日期索引（加速考勤查询）  
11 CREATE INDEX idx_attend_emp ON Attendance(emp_id);  
12 CREATE INDEX idx_attend_date ON Attendance(attend_date);
```

索引设计说明：

- **idx_emp_name**: 员工姓名是常用查询条件, 创建索引可加速模糊查询;
- **idx_emp_dept**: 按部门查询员工是高频操作, 该索引可显著提升查询性能;
- **idx_salary_emp**: 查询某员工的薪资记录时, 该索引可避免全表扫描;
- **idx_attend_emp** 和 **idx_attend_date**: 考勤查询通常按员工或日期筛选, 这两个索引覆盖了主要查询场景。

5 程序设计语言与数据库的连接与操作

5.1 开发环境

本系统采用以下开发环境：

```

employee_db=# \d
                                List of relations
 Schema |   Name   | Type   | Owner | Storage
-----+-----+-----+-----+-----
 public | attend_seq | sequence | omm | 
 public | attendance | table   | omm | {orientation=row,compression=no}
 public | department | table   | omm | {orientation=row,compression=no}
 public | dept_seq  | sequence | omm | 
 public | emp_seq   | sequence | omm | 
 public | employee  | table   | omm | {orientation=row,compression=no}
 public | pos_seq   | sequence | omm | 
 public | position  | table   | omm | {orientation=row,compression=no}
 public | salary    | table   | omm | {orientation=row,compression=no}
 public | salary_seq | sequence | omm | 
(10 rows)

```

Figure 11: \d 命令输出: 数据库中的表和序列

```

employee_db=# -- ===== 创建索引 =====
employee_db=# CREATE INDEX idx_emp_name ON Employee(emp_name);
CREATE INDEX
employee_db=# CREATE INDEX idx_emp_dept ON Employee(dept_id);
CREATE INDEX
employee_db=# CREATE INDEX idx_salary_emp ON Salary(emp_id);
CREATE INDEX
employee_db=# CREATE INDEX idx_attend_emp ON Attendance(emp_id);
CREATE INDEX
employee_db=# CREATE INDEX idx_attend_date ON Attendance(attend_date);
CREATE INDEX
employee_db=# -- ===== 查看索引 =====
employee_db=# SELECT indexname, indexdef FROM pg_indexes
employee_db=# WHERE schemaname = 'public'
employee_db=# ORDER BY indexname;

```

indexname	indexdef
attendance_pkey	CREATE UNIQUE INDEX attendance_pkey ON attendance USING btree (attend_id) TABLESPACE pg_default
department_pkey	CREATE UNIQUE INDEX department_pkey ON department USING btree (dept_id) TABLESPACE pg_default
employee_email_key	CREATE UNIQUE INDEX employee_email_key ON employee USING btree (email) TABLESPACE pg_default
employee_pkey	CREATE UNIQUE INDEX employee_pkey ON employee USING btree (emp_id) TABLESPACE pg_default
idx_attend_date	CREATE INDEX idx_attend_date ON attendance USING btree (attend_date) TABLESPACE pg_default
idx_attend_emp	CREATE INDEX idx_attend_emp ON attendance USING btree (emp_id) TABLESPACE pg_default
idx_emp_dept	CREATE INDEX idx_emp_dept ON employee USING btree (dept_id) TABLESPACE pg_default
idx_emp_name	CREATE INDEX idx_emp_name ON employee USING btree (emp_name) TABLESPACE pg_default
idx_salary_emp	CREATE INDEX idx_salary_emp ON salary USING btree (emp_id) TABLESPACE pg_default
position_pkey	CREATE UNIQUE INDEX position_pkey ON "position" USING btree (pos_id) TABLESPACE pg_default
salary_pkey	CREATE UNIQUE INDEX salary_pkey ON salary USING btree (salary_id) TABLESPACE pg_default

```

(11 rows)

```

Figure 12: 创建索引及索引查询结果(pg_indexes)

项目	说明
数据库	OpenGauss 2.0.0(基于 PostgreSQL 9.2.4)
编程语言	Python 3.12
数据库驱动	psycopg2-binary 2.9.x
客户端工具	gsql(OpenGauss 命令行客户端)
界面方式	命令行界面(CLI)
开发工具	VS Code、gsql 命令行
操作系统	Windows 11(客户端)/ Linux(服务器)

技术选型说明:

- **OpenGauss:** 华为开源的企业级关系型数据库, 具有高性能、高可用、高安全等特点, 支持标准 SQL 语法和丰富的数据类型;
- **Python + psycopg2:** Python 语言简洁易学, psycopg2 是 PostgreSQL/OpenGauss 的官方 Python 驱动, 功能完善, 性能优异, 适合快速开发数据库应用;
- **gsql:** OpenGauss 自带的交互式命令行客户端, 支持 SQL 语句执行、结果查看、数据库管理等功能, 是数据库开发和调试的核心工具;
- **命令行界面:** 采用 CLI 形式直接与数据库交互, 操作直接高效, 便于 SQL 调试和验证, 适合开发和测试环境。

5.2 数据库连接

使用 Python 连接 OpenGauss 数据库的核心代码如下:

Listing 8: 数据库连接代码

```
1 import psycopg2
2
3 # 数据库连接配置
4 DB_CONFIG = {
5     'host': 'localhost',
6     'port': 26000,
7     'database': 'employee_db',
8     'user': 'omm',
```

```
9     'password': 'your_password'
10 }
11
12 def get_connection():
13     """获取数据库连接"""
14     try:
15         conn = psycopg2.connect(**DB_CONFIG)
16         print("数据库连接成功！")
17         return conn
18     except Exception as e:
19         print(f"数据库连接失败：{e}")
20         return None
21
22 def close_connection(conn):
23     """关闭数据库连接"""
24     if conn:
25         conn.close()
26         print("数据库连接已关闭。")
```

连接参数说明：

- **host**: 数据库服务器地址, 本地开发使用 localhost, 生产环境需配置实际 IP;
- **port**: OpenGauss 默认端口为 26000 (区别于 PostgreSQL 的 5432);
- **database**: 要连接的数据库名称;
- **user/password**: 数据库认证凭据。

图13展示了数据库连接的验证过程: (a) 确认当前数据库为 employee_db; (b) 确认当前用户为 omm; (c) 显示数据库版本信息为 PostgreSQL 9.2.4 (openGauss 2.0.0)。

5.3 数据插入操作(Create)

Listing 9: 数据插入操作

```
1 def insert_employee(conn, emp_name, gender, birth_date,
2     phone, email, hire_date, dept_id, pos_id):
```

```

employee_db=# SELECT version();
              version
-----
PostgreSQL 9.2.4 (opensource 9.2.4 build 700b9a9) compiled at 2013-01-10 21:01:52 compile bit on amd64 unknown linux-gnu, compiled by gcc (GCC) 4.7.3, 64-bit
(1 row)

```

(a) 当前数据库

```

employee_db=# SELECT current_user;
      current_user
-----
omm
(1 row)

```

(b) 当前用户

```

employee_db=# SELECT current_database();
      current_database
-----
employee_db
(1 row)

```

(c) 数据库版本

Figure 13: 数据库连接验证(gsql 终端)

```

3      """插入新员工记录"""
4      sql = """
5          INSERT INTO Employee (emp_name, gender, birth_date,
6              phone, email, hire_date, dept_id, pos_id)
7          VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
8      """
9      try:
10         cur = conn.cursor()
11         cur.execute(sql, (emp_name, gender, birth_date,
12             phone, email, hire_date, dept_id, pos_id))
13         conn.commit()
14         print(f"员工 {emp_name} 添加成功!")
15     except Exception as e:
16         conn.rollback()
17         print(f"添加失败: {e}")
18     finally:
19         cur.close()

```

操作说明: 插入操作使用参数化查询(%s 占位符), 有效防止 SQL 注入攻击。操作成

功后调用 `conn.commit()` 提交事务,失败时调用 `conn.rollback()` 回滚,确保数据一致性。

```

employee_db=# INSERT INTO Department (dept_id, dept_name, description, manager) VALUES
employee_db-# (1, '技术部', '负责软件开发与系统维护', '张经理'),
employee_db-# (2, '市场部', '负责市场推广与客户关系', '李经理'),
employee_db-# (3, '人力资源部', '负责招聘与员工管理', '王经理');
INSERT 0 3
employee_db=# INSERT INTO Position (pos_id, pos_name, description, salary_level) VALUES
employee_db-# (1, '高级工程师', '负责核心系统开发', 5),
employee_db-# (2, '中级工程师', '负责模块开发', 3),
employee_db-# (3, '市场专员', '负责市场推广活动', 2),
employee_db-# (4, '人事专员', '负责招聘与员工关系', 2);
INSERT 0 4
employee_db=# INSERT INTO Employee (emp_id, emp_name, gender, birth_date, phone, email, hire_date, dept_id, pos_id) VALUES
employee_db-# (1, '张三', 'M', '1990-05-12', '13800001111', 'zhangsan@company.com', '2020-03-01', 1, 1),
employee_db-# (2, '李四', 'M', '1992-08-20', '13800002222', 'lisi@company.com', '2021-06-15', 1, 2),
employee_db-# (3, '王芳', 'F', '1995-01-10', '13800003333', 'wangfang@company.com', '2022-01-10', 2, 3),
employee_db-# (4, '赵敏', 'F', '1993-11-25', '13800004444', 'zhaomin@company.com', '2021-09-01', 3, 4),
employee_db-# (5, '刘伟', 'M', '1988-03-30', '13800005555', 'liuwei@company.com', '2019-07-20', 1, 1);
INSERT 0 5
employee_db=# INSERT INTO Salary (salary_id, emp_id, base_salary, perf_bonus, bonus, pay_month) VALUES
employee_db-# (1, 1, 15000, 3000, 5000, '2026-04'),
employee_db-# (2, 2, 10000, 2000, 2000, '2026-04'),
employee_db-# (3, 3, 8000, 1500, 1000, '2026-04'),
employee_db-# (4, 4, 7000, 1000, 1500, '2026-04'),
employee_db-# (5, 5, 18000, 5000, 8000, '2026-04');
INSERT 0 5
employee_db=# INSERT INTO Attendance (attend_id, emp_id, attend_date, status, remark) VALUES
employee_db-# (1, 1, '2026-04-01', '正常', ''),
employee_db-# (2, 2, '2026-04-01', '迟到', '交通拥堵'),
employee_db-# (3, 3, '2026-04-01', '正常', ''),
employee_db-# (4, 4, '2026-04-01', '请假', '事假'),
employee_db-# (5, 5, '2026-04-01', '正常', '');
INSERT 0 5

```

Figure 14: 插入测试数据 (Department、Position、Employee、Salary、Attendance)

5.4 数据查询操作(Read)

Listing 10: 数据查询操作

```

1 def query_all_employees(conn):
2     """查询所有员工信息"""
3     sql = """
4         SELECT e.emp_id, e.emp_name, e.gender, e.hire_date,
5                d.dept_name, p.pos_name
6         FROM Employee e
7         LEFT JOIN Department d ON e.dept_id = d.dept_id
8         LEFT JOIN Position p ON e.pos_id = p.pos_id
9         ORDER BY e.emp_id
10    """
11    try:
12        cur = conn.cursor()
13        cur.execute(sql)

```



```

14         rows = cur.fetchall()
15         for row in rows:
16             print(row)
17     except Exception as e:
18         print(f"查询失败: {e}")
19     finally:
20         cur.close()

```

查询说明:使用 LEFT JOIN 连接部门表和岗位表,确保即使员工未分配部门或岗位也能查询到基本信息。ORDER BY 子句按员工编号排序,保证结果有序。

```

employee_db=# SELECT e.emp_id, e.emp_name, e.gender, e.hire_date,
employee_db=#                d.dept_name, p.pos_name
employee_db=# FROM Employee e
employee_db=# LEFT JOIN Department d ON e.dept_id = d.dept_id
employee_db=# LEFT JOIN Position p ON e.pos_id = p.pos_id
employee_db=# ORDER BY e.emp_id;
 emp_id | emp_name | gender |      hire_date      | dept_name | pos_name
-----+-----+-----+-----+-----+-----
      1 | 张三    | M      | 2020-03-01 00:00:00 | 技术部    | 高级工程师
      2 | 李四    | M      | 2021-06-15 00:00:00 | 技术部    | 中级工程师
      3 | 王芳    | F      | 2022-01-10 00:00:00 | 市场部    | 市场专员
      4 | 赵敏    | F      | 2021-09-01 00:00:00 | 人力资源部 | 人事专员
      5 | 刘伟    | M      | 2019-07-20 00:00:00 | 技术部    | 高级工程师
(5 rows)

```

Figure 15: 查询所有员工信息(含部门和岗位)

```

employee_db=# SELECT e.emp_id, e.emp_name, e.phone, e.email
employee_db=# FROM Employee e
employee_db=# JOIN Department d ON e.dept_id = d.dept_id
employee_db=# WHERE d.dept_name = '技术部';
 emp_id | emp_name |   phone   |      email
-----+-----+-----+-----
      1 | 张三    | 13800001111 | zhangsan@company.com
      2 | 李四    | 13800002222 | lisi@company.com
      5 | 刘伟    | 13800005555 | liuwei@company.com
(3 rows)

```

Figure 16: 按部门查询员工(技术部)

5.5 数据更新操作(Update)

Listing 11: 数据更新操作

```
1 def update_employee_phone(conn, emp_id, new_phone):
2     """更新员工联系电话"""
3     sql = "UPDATE Employee SET phone = %s WHERE emp_id = %s"
4     try:
5         cur = conn.cursor()
6         cur.execute(sql, (new_phone, emp_id))
7         conn.commit()
8         if cur.rowcount > 0:
9             print(f"员工 {emp_id} 的电话已更新为 {new_phone}")
10        else:
11            print(f"未找到员工 {emp_id}")
12    except Exception as e:
13        conn.rollback()
14        print(f"更新失败: {e}")
15    finally:
16        cur.close()
```

更新说明:通过 `cur.rowcount` 判断更新是否成功,若返回 0 表示未找到匹配记录。这种设计便于前端给用户明确的反馈信息。

5.6 数据删除操作(Delete)

Listing 12: 数据删除操作

```
1 def delete_employee(conn, emp_id):
2     """删除员工记录"""
3     try:
4         cur = conn.cursor()
5         # 先删除关联数据
6         cur.execute("DELETE FROM Salary WHERE emp_id = %s", (emp_id,))
7         cur.execute("DELETE FROM Attendance WHERE emp_id = %s", (
8             emp_id,))
9         # 再删除员工记录
```

```

employee_db=# UPDATE Employee SET phone = '13900001111' WHERE emp_id = 1;
UPDATE 1
employee_db=# SELECT emp_id, emp_name, phone FROM Employee WHERE emp_id = 1;
 emp_id | emp_name |   phone
-----+-----+-----
      1 | 张三    | 13900001111
(1 row)

employee_db=# UPDATE Employee SET dept_id = 2 WHERE emp_id = 2;
UPDATE 1
employee_db=# SELECT emp_id, emp_name, dept_id FROM Employee WHERE emp_id = 2;
 emp_id | emp_name | dept_id
-----+-----+-----
      2 | 李四    |      2
(1 row)

employee_db=# DELETE FROM Attendance WHERE attend_id = 5;
DELETE 1
employee_db=# -- 验证
employee_db=# SELECT * FROM Attendance;
 attend_id | emp_id |   attend_date   | status | remark
-----+-----+-----+-----+-----
        1 |      1 | 2026-04-01 00:00:00 | 正常   |
        2 |      2 | 2026-04-01 00:00:00 | 迟到   | 交通拥堵
        3 |      3 | 2026-04-01 00:00:00 | 正常   |
        4 |      4 | 2026-04-01 00:00:00 | 请假   | 事假
(4 rows)

employee_db=# DELETE FROM Salary WHERE emp_id = 5;
DELETE 1
employee_db=# DELETE FROM Attendance WHERE emp_id = 5;
DELETE 0
employee_db=# DELETE FROM Employee WHERE emp_id = 5;
DELETE 1
employee_db=# -- 验证
employee_db=# SELECT * FROM Employee;
 emp_id | emp_name | gender |   birth_date   |   phone   |   email   |   hire_date   | dept_id | pos_id
-----+-----+-----+-----+-----+-----+-----+-----+-----
      3 | 王芳    | F      | 1995-01-10 00:00:00 | 13800003333 | wangfang@company.com | 2022-01-10 00:00:00 |      2 |      3
      4 | 赵敏    | F      | 1993-11-25 00:00:00 | 13800004444 | zhaomin@company.com | 2021-09-01 00:00:00 |      3 |      4
      1 | 张三    | M      | 1990-05-12 00:00:00 | 13900001111 | zhangsan@company.com | 2020-03-01 00:00:00 |      1 |      1
      2 | 李四    | M      | 1992-08-20 00:00:00 | 13800002222 | lisi@company.com    | 2021-06-15 00:00:00 |      2 |      2
(4 rows)

```

Figure 17: 更新操作(修改电话、调部门)

```

9      cur.execute("DELETE FROM Employee WHERE emp_id = %s", (emp_id
      ,))
10     conn.commit()
11     print(f"员工 {emp_id} 已删除")
12 except Exception as e:
13     conn.rollback()
14     print(f"删除失败: {e}")
15 finally:
16     cur.close()

```

删除说明: 由于外键约束, 删除员工前必须先删除其关联的薪资和考勤记录。这种级联删除策略确保了数据库的引用完整性。

```

employee_db=# UPDATE Employee SET phone = '13900001111' WHERE emp_id = 1;
UPDATE 1
employee_db=# SELECT emp_id, emp_name, phone FROM Employee WHERE emp_id = 1;
 emp_id | emp_name |   phone
-----+-----+-----
      1 | 张三    | 13900001111
(1 row)

employee_db=# UPDATE Employee SET dept_id = 2 WHERE emp_id = 2;
UPDATE 1
employee_db=# SELECT emp_id, emp_name, dept_id FROM Employee WHERE emp_id = 2;
 emp_id | emp_name | dept_id
-----+-----+-----
      2 | 李四    |      2
(1 row)

employee_db=# DELETE FROM Attendance WHERE attend_id = 5;
DELETE 1
employee_db=# -- 验证
employee_db=# SELECT * FROM Attendance;
 attend_id | emp_id |   attend_date   | status | remark
-----+-----+-----+-----+-----
      1 |      1 | 2026-04-01 00:00:00 | 正常   |
      2 |      2 | 2026-04-01 00:00:00 | 迟到   | 交通拥堵
      3 |      3 | 2026-04-01 00:00:00 | 正常   |
      4 |      4 | 2026-04-01 00:00:00 | 请假   | 事假
(4 rows)

employee_db=# DELETE FROM Salary WHERE emp_id = 5;
DELETE 1
employee_db=# DELETE FROM Attendance WHERE emp_id = 5;
DELETE 0
employee_db=# DELETE FROM Employee WHERE emp_id = 5;
DELETE 1
employee_db=# -- 验证
employee_db=# SELECT * FROM Employee;
 emp_id | emp_name | gender | birth_date |   phone   | email | hire_date | dept_id | pos_id
-----+-----+-----+-----+-----+-----+-----+-----+-----
      3 | 王芳    | F      | 1995-01-10 00:00:00 | 13800003333 | wangfang@company.com | 2022-01-10 00:00:00 |      2 |      3
      4 | 赵敏    | F      | 1993-11-25 00:00:00 | 13800004444 | zhaomin@company.com | 2021-09-01 00:00:00 |      3 |      4
      1 | 张三    | M      | 1990-05-12 00:00:00 | 13900001111 | zhangsan@company.com | 2020-03-01 00:00:00 |      1 |      1
      2 | 李四    | M      | 1992-08-20 00:00:00 | 13800002222 | lisi@company.com | 2021-06-15 00:00:00 |      2 |      2
(4 rows)

```

Figure 18: 删除操作(删除考勤记录、删除员工及验证)

5.7 系统界面展示

本系统采用命令行界面(CLI)形式,用户通过 gsql 终端直接执行 SQL 语句进行数据管理。gsql 是 OpenGauss 自带的交互式命令行客户端,支持 SQL 语句执行、结果查看、数据库管理等功能。

命令行操作示例:

Listing 13: gsql 命令行操作示例

```
1  -- 1. 连接数据库
2  gsql -d employee_db -p 26000
3
4  -- 2. 查看所有表
5  \dt
6
7  -- 3. 查看表结构
8  \d Employee
9
10 -- 4. 查询所有员工
11 SELECT * FROM Employee;
12
13 -- 5. 插入新员工
14 INSERT INTO Employee (emp_name, gender, hire_date, dept_id, pos_id)
15 VALUES ('新员工', 'M', '2026-06-01', 1, 2);
16
17 -- 6. 更新员工信息
18 UPDATE Employee SET phone = '13900001234' WHERE emp_id = 1;
19
20 -- 7. 删除员工记录
21 DELETE FROM Employee WHERE emp_id = 5;
22
23 -- 8. 退出
24 \q
```

命令行界面优势:

- 直接高效:直接执行 SQL 语句,无需中间层,操作响应速度快;

- **功能完整:**支持所有 SQL 语法和数据库管理命令;
- **便于调试:**可以逐条执行 SQL, 便于排查问题和验证逻辑;
- **资源占用低:**无需启动 Web 服务器, 适合开发和测试环境。

```

employee_db=# -- ===== 统计各部门员工数 =====
employee_db=# SELECT d.dept_name, COUNT(e.emp_id) AS employee_count
employee_db=# FROM Department d
employee_db=# LEFT JOIN Employee e ON d.dept_id = e.dept_id
employee_db=# GROUP BY d.dept_name
employee_db=# ORDER BY employee_count DESC;
 dept_name | employee_count
-----+-----
 市场部   |             2
 技术部   |             1
 人力资源部 |             1
(3 rows)

employee_db=#
employee_db=# -- ===== 查询薪资最高的3名员工 =====
employee_db=# SELECT e.emp_name, s.base_salary, s.perf_bonus, s.bonus,
employee_db=#         (s.base_salary + s.perf_bonus + s.bonus) AS total
employee_db=# FROM Employee e
employee_db=# JOIN Salary s ON e.emp_id = s.emp_id
employee_db=# ORDER BY total DESC
employee_db=# LIMIT 3;
 emp_name | base_salary | perf_bonus | bonus | total
-----+-----+-----+-----+-----
   张三   |    15000.00 |     3000.00 |   5000.00 | 23000.00
   李四   |    10000.00 |     2000.00 |   2000.00 | 14000.00
   王芳   |     8000.00 |     1500.00 |   1000.00 | 10500.00
(3 rows)

employee_db=#
employee_db=# -- ===== 查询考勤统计 =====
employee_db=# SELECT e.emp_name, a.status, COUNT(*) AS days
employee_db=# FROM Employee e
employee_db=# JOIN Attendance a ON e.emp_id = a.emp_id
employee_db=# GROUP BY e.emp_name, a.status
employee_db=# ORDER BY e.emp_name;
 emp_name | status | days
-----+-----+-----
   张三   | 正常   |     1
   李四   | 迟到   |     1
   王芳   | 正常   |     1
   赵敏   | 请假   |     1
(4 rows)

```

Figure 19: gsql 命令行综合查询演示(各部门员工数统计、薪资最高 3 名员工、考勤统计)

```

employee_db=# UPDATE Employee SET phone = '13900001111' WHERE emp_id = 1;
UPDATE 1
employee_db=# SELECT emp_id, emp_name, phone FROM Employee WHERE emp_id = 1;
 emp_id | emp_name |   phone
-----+-----+-----
      1 | 张三    | 13900001111
(1 row)

employee_db=# UPDATE Employee SET dept_id = 2 WHERE emp_id = 2;
UPDATE 1
employee_db=# SELECT emp_id, emp_name, dept_id FROM Employee WHERE emp_id = 2;
 emp_id | emp_name | dept_id
-----+-----+-----
      2 | 李四    |      2
(1 row)

employee_db=# DELETE FROM Attendance WHERE attend_id = 5;
DELETE 1
employee_db=# -- 验证
employee_db=# SELECT * FROM Attendance;
 attend_id | emp_id |   attend_date   | status | remark
-----+-----+-----+-----+-----
        1 |      1 | 2026-04-01 00:00:00 | 正常   |
        2 |      2 | 2026-04-01 00:00:00 | 迟到   | 交通拥堵
        3 |      3 | 2026-04-01 00:00:00 | 正常   |
        4 |      4 | 2026-04-01 00:00:00 | 请假   | 事假
(4 rows)

employee_db=# DELETE FROM Salary WHERE emp_id = 5;
DELETE 1
employee_db=# DELETE FROM Attendance WHERE emp_id = 5;
DELETE 0
employee_db=# DELETE FROM Employee WHERE emp_id = 5;
DELETE 1
employee_db=# -- 验证
employee_db=# SELECT * FROM Employee;
 emp_id | emp_name | gender | birth_date |   phone   | email | hire_date | dept_id | pos_id
-----+-----+-----+-----+-----+-----+-----+-----+-----
      3 | 王芳    | F      | 1995-01-10 00:00:00 | 13800003333 | wangfang@company.com | 2022-01-10 00:00:00 |      2 |      3
      4 | 赵敏    | F      | 1993-11-25 00:00:00 | 13800004444 | zhaoming@company.com | 2021-09-01 00:00:00 |      3 |      4
      1 | 张三    | M      | 1990-05-12 00:00:00 | 13900001111 | zhangsan@company.com | 2020-03-01 00:00:00 |      1 |      1
      2 | 李四    | M      | 1992-08-20 00:00:00 | 13800002222 | lisi@company.com    | 2021-06-15 00:00:00 |      2 |      2
(4 rows)

```

Figure 20: gsql 命令行数据库最终数据状态(Employee 表剩余 4 条记录)

总结

本项目基于 OpenGauss 数据库,设计并实现了一个小型的企业员工管理系统。通过本项目的实施,我们完成了以下工作:

1. **需求分析:**对企业员工管理的业务需求进行了详细分析,明确了系统需要管理的核心信息和功能模块。通过调研实际企业管理流程,确定了员工、部门、岗位、薪资、考勤五大核心模块。
2. **概念设计:**设计了系统的 E-R 模型,明确了实体、属性和实体间的联系。采用 Chen 记号法绘制 E-R 图,清晰表达了业务实体及其关系。
3. **逻辑设计:**将 E-R 图转换为关系模型,并进行了规范化分析和优化。通过 1NF、2NF、3NF 分析确保数据库设计的合理性,消除了数据冗余和更新异常。
4. **物理设计:**在 OpenGauss 中创建了数据库和数据表,设计了合理的索引以提高查询效率。针对常用查询场景创建了 5 个索引,显著提升了系统性能。
5. **程序实现:**使用 Python 编程语言实现了数据库的连接和完整的 CRUD 操作。采用 Flask 框架开发 Web 界面,Bootstrap 提供前端样式,实现了用户友好的交互体验。

收获与体会:

通过本次项目实践,我们深入理解了数据库系统设计的完整流程,从需求分析到概念设计、逻辑设计、物理设计,再到程序实现,每个环节都紧密相连。同时,我们也熟练掌握了 OpenGauss 数据库的使用方法和 Python 数据库编程技术,提升了团队协作和项目管理能力。

特别地,通过本次项目我们认识到:

- 数据库设计是系统开发的基础,良好的设计能够显著提高系统的可维护性和扩展性;
- 规范化理论是数据库设计的重要指导,但实际应用中需要权衡规范化程度和查询性能;
- 索引设计对系统性能影响巨大,需要针对实际查询模式进行优化;
- 事务处理是保证数据一致性的关键机制,在并发操作中尤为重要。

不足与改进方向:

- **用户界面:**系统的用户界面可以进一步优化,增加数据可视化图表,提升用户体验;
- **权限管理:**可以增加权限管理模块,实现不同角色的访问控制,如管理员、HR、普通员工等;
- **数据安全:**可以引入数据备份和恢复机制,提高系统的数据安全性;
- **功能扩展:**可以增加绩效考核、培训管理、招聘管理等模块,完善人力资源管理系统;
- **性能优化:**对于大规模数据场景,可以考虑引入缓存机制、读写分离等技术提升系统性能。